DS Written Homework 2

Student ID: b10201054

April 10, 2025

1 Binary Tree

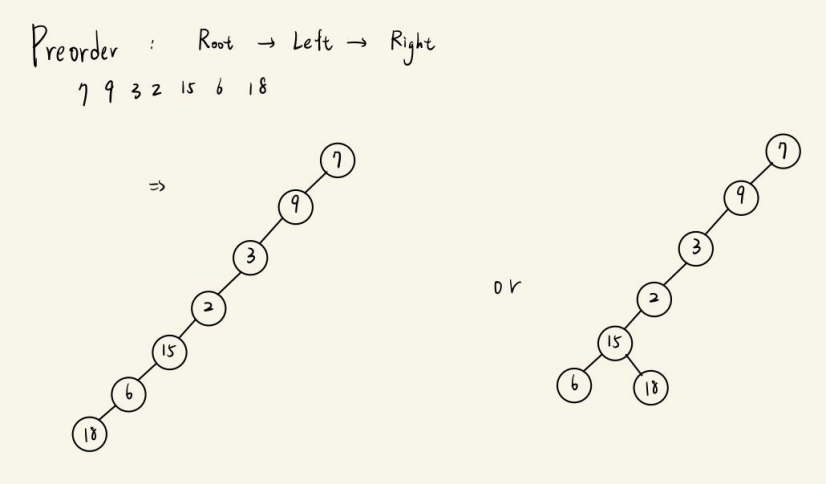

Figure 1: Binarytree

(a)

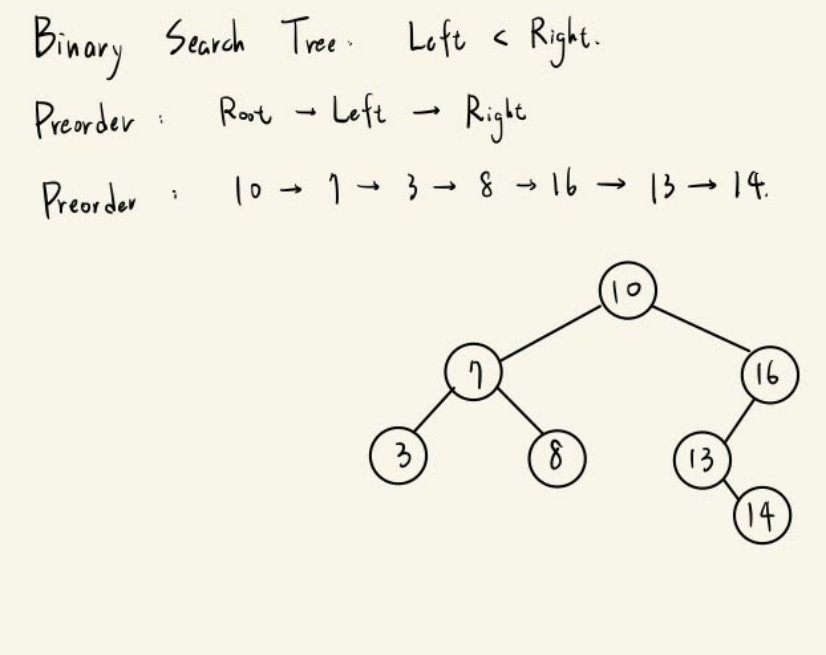

Figure 2: Binarytree

(b)

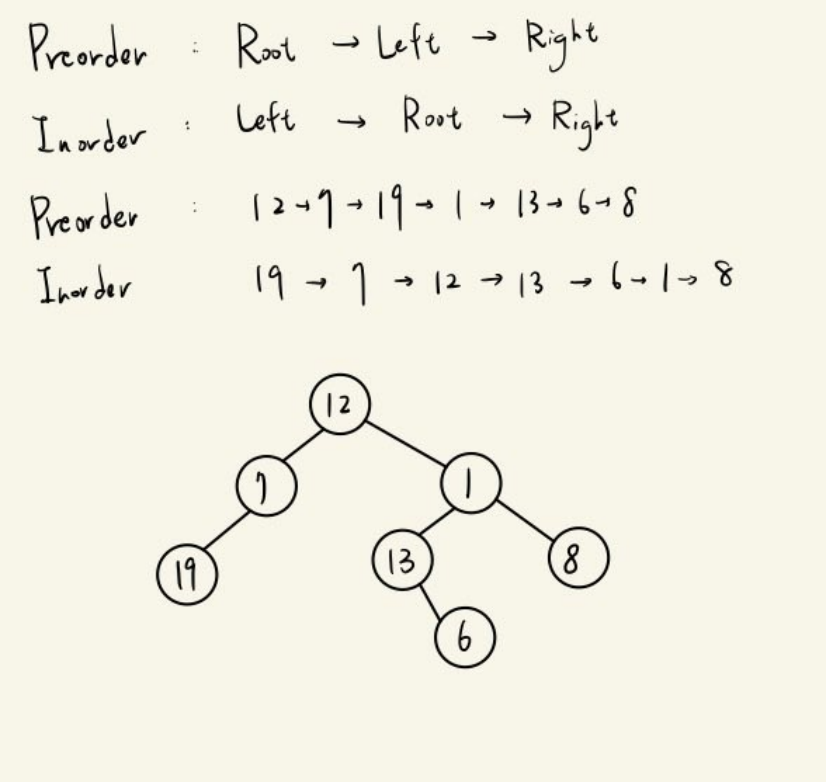

Figure 3: Binarytree

(c)

2 Recursion Problem

```python
# Function to calculate total sodas
def countSodas(money, price, bottle):
    sodas = money // price
    return sodas + countRec(sodas, bottle)

# Recursive helper function
def countRec(sodas, bottle):
    if sodas < bottle:
        return 0
    newSodas = sodas // bottle
    return newSodas + countRec(sodas % bottle + newSodas, bottle)

# Example usage
money = 15
price = 1
bottle = 3
print(countSodas(money, price, bottle))
```

Figure 4: Binarytree

(a)

(b) Note that the recurrence occurs in countRec function. We want to find that how much times the function runs.

$$countRec(sodas, bottle) = countRec(sodas\%bottle + sodas//bottle, bottle) + O(1)$$

If $sodas < bottle$, the recurrence ends. The primitive number of soda is $\frac{M}{P}$. Consider the equation $\frac{M}{P} = \sum_{k=1}^{n} a_k B^k + r_1$ with $a_k < B, r < B$.

The second input is $\sum_{k=1}^{n-1} a_{k+1} B^k + b_1 B + r_2$ where $b_1 B + r_2 = a_1 + r_1$.

The third input is $\sum_{k=1}^{n-2} a_{k+2} B^k + b_2 B + r_3$ where $b_2 B + r_3 = a_2 + b_1 + r_2$.

Then we can find the n-th input is $a_n + b_{n-1} B + r_n$ where $b_{n-1} B + r_n = a_{n-1} + b_{n-2} + r_{n-1}$.

Note that $a_n, b_{n-1}, r_n < B$, there are some choice:

1. $a_n + b_{n-1} + r_n < B$: The function runs n times.
2. $a_n + b_{n-1} + r_n = mB + k$ but $m + k < B$: The function runs n+1 times.
3. $a_n + b_{n-1} + r_n = mB + k$ and $m + k > B$: The function runs n+2 times.

Hence the time complexity is decided by n, and

$$n = \left\lfloor \log_B(\frac{M}{P}) \right\rfloor$$

The time complexity is $O(\log_B(\frac{M}{P}))$.

3 AVL Tree

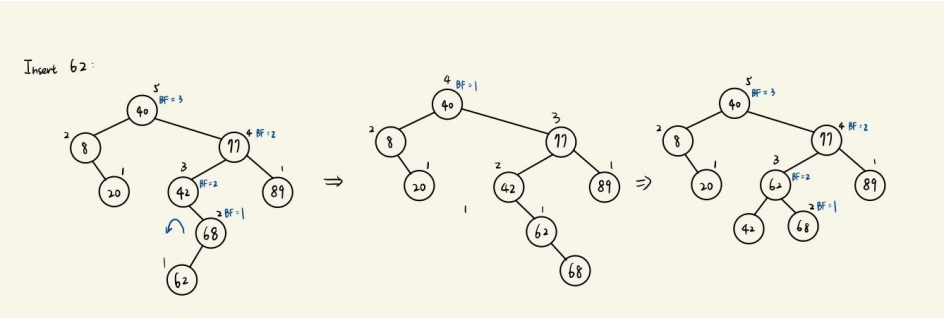

Figure 5: Binarytree

(a)

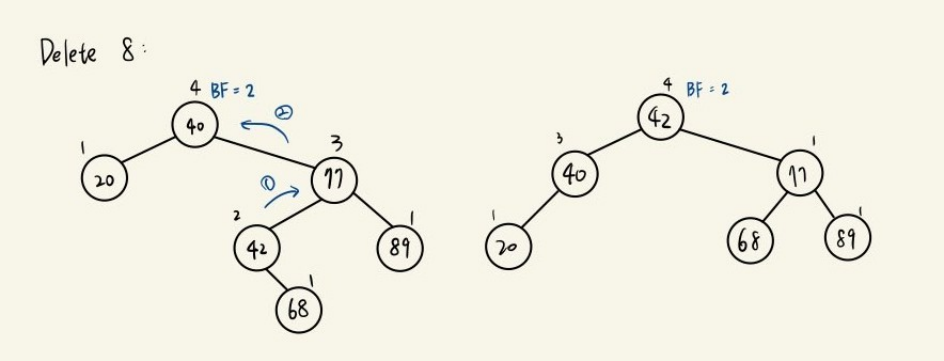

Figure 6: Binarytree

(b)

(c) The minimum height is $\lceil(\log_2(n+1))\rceil$.

Since AVL Tree must follow the rule $|left\_height - right\_height| \leq 1$, the worst case of AVL Tree is Fibonacci Tree. Fibonacci tree gives us $T(0) = 0, T(1) = 1, T(h) = T(h-1) + T(h-2) + 1$, where h means the tree's height, and $T(h)$ means the number of nodes in the h-height Fibonacci Tree. In this recurrence relation, we can easily compute the general function $T(h) = \frac{(\phi^{h+2} - \varphi^{h+2})}{\sqrt{5}} - 1$, where ϕ is golden ratio and φ is its conjugate.

In this case, we want to find h such that $T(h) = 493$. We can set $\varphi^{h+2} = \epsilon$ since it's much more smaller than ϕ^{h+2}. Then the equation will become

$$T(h) = \frac{\phi^{h+2}}{\sqrt{5}} - \epsilon - 1 = 493$$

After solving the equation, we have $h = \lfloor \log_\phi \sqrt{5}(494 + \epsilon) - 2 \rfloor$. After computing the $\lfloor \log_\phi \sqrt{5}(494 + \epsilon) - 2 \rfloor = 12$ and $\lceil(\log_2(494))\rceil = 9$, the possible heights is 9, 10, 11, 12.

4 Parentheses Balance Problem

(a) I think stack is the great data structure to confirm whether the parentheses are balanced. If we want to confirm whether the parentheses are balanced, we need to check if some illegal mark is inside of the small order ones. The pop function in stack is to pop the last element, so that we can know now we are in which order. If the order of the next element we popped is not illegal, we can find it easily.

(b) 15 permutations. {}[](), {}()[], []{}(), [](){}, (){}[], ()[]{}
{[]}(), (){[]}, {()}[], []{()}, {}[()], [()]{}
{[]()}, {()[]}, {[()]}

(c) Consider the order : (1), [2], {3}. Now save the permutations as a stack.

1. Pop, save the element.
   Record its minimum order and each number appear times.
2. Pop.
   If the element is ( ),],}) check if the element order is equal or less than the minimum order. If it's legal, record its order in list and its appear times+1.
   If the element is ((, [, {), the appear times of its order-1, then update its minimum order.

5 Complexity Analysis

(a) Consider the recurrence relation

$$\begin{cases} a_n &= a_{n-2} + n \\ a_1 &= k_1(constant) \\ a_0 &= k_0(constant) \end{cases}$$

If n is even, we have $a_n = a_0 + \sum_{k=1}^{\frac{n}{2}} k = a_0 + \frac{1}{2} \cdot \frac{n}{2} \cdot \frac{n+2}{2}$. Hence the complexity is $O(n^2)$.

(b) Consider the recurrence relation

$$\begin{cases} a_n &= 2a_{n-1} + n \\ a_1 &= k(constant) \end{cases}$$

We have the general $a_n = 2^{n-1}(a_1 + 3)$, which means the complexity is $O(2^n)$

(c) Consider the recurrence relation

$$\begin{cases} (a_n + \frac{1}{4}) &= 2(a_{n-1} + \frac{1}{4}) + 3(a_{n-2} + \frac{1}{4}) \\ a_1 &= 1 \\ a_2 &= 1 \end{cases}$$

After solving its characteristic equation $r^2 = 2r + 3$, we get two roots 3, -1. The general $a_n = C \cdot (3)^n + D \cdot (-1)^n$. Hence the complexity is $O(3^n)$

(d) Consider the recurrence relation

$$\begin{cases} a_n &= 8a_{\frac{n}{4}} + \log n \\ a_1 &= k(constant) \end{cases}$$

The recurrence relation will give us $a_n = 8^{\log_4 n} a_1 + O(\log n) = n^{\frac{3}{2}} + O(\log n)$. Since we know polynomial is growing faster than log, $T(n)$ has time complexity $O(n^{\frac{3}{2}})$.

(e) $T(n) = T(\sqrt{n}) + 1$, assume that when m-th recurrence, $n^{\frac{1}{2^m}} \approx 1 + 10^{-6}$. In the m-th recurrence we have

$$T(n) = T(n^{\frac{1}{2^m}}) + m \approx m + 1$$

Compute m:

$$\log n = 2^m \log(1 + 10^6) \Rightarrow m = \log_2(\frac{\log n}{\log(1 + 10^{-6})})$$

Hence, the complexity is $O(\log \log n)$

(f) $T(n) = 3T(\sqrt{n}) + 1$, assume that when m-th recurrence, $n^{\frac{1}{2^m}} \approx 1 + 10^{-6}$. In the m-th recurrence we have

$$T(n) = 3^m \cdot T(n^{\frac{1}{2^m}}) + m \approx m + 3^m$$

Compute m:

$$\log n = 2^m \log(1 + 10^6) \Rightarrow m = \log_2(\frac{\log n}{\log(1 + 10^{-6})})$$

Then 3^m:

$$3^{\log_2(\frac{\log n}{\log(1+10^{-6})})} = \log_2 3 \frac{\log n}{\log(1 + 10^{-6})}$$

Hence, the complexity is $O(\log n)$

6 Problem 6

In-order traversal will follow the rule (Left child tree $\rightarrow$ Root $\rightarrow$ Right child tree).

$\Rightarrow$ part: We call the BST T. T has the structure $L - x - R$ with $L < x < R$. By the definition of in-order traversal, it follows $L \rightarrow x \rightarrow R$. Then it is sorted in ascending order.

$\Leftarrow$ part: In-order traversal is sorted in ascending order. With in-order traversal, we can struct a tree T with $L - x - R$. Since in-order traversal is sorted in ascending order, we have $L < x < R$. Hence the tree T is a binary search tree.